

SRM Project — Independent Validation Report

Andrew · Group 13 · MCTR908 Electric Drives · Spring 2026

Generated: April 28, 2026

1 What's the Problem?

For our Electric Drives project, we built a full closed-loop speed controller for a **6/4 Switched Reluctance Motor (SRM)** in Simulink. The system has three layers:

- **Speed loop** — a PI controller that takes the speed error ($\omega^* - \omega$) and outputs a torque demand T^*
- **Torque distribution** — a Cubic Torque Sharing Function (TSF) that splits T^* among three phases to minimise torque ripple during commutation
- **Current loop** — a hysteresis controller per phase that switches the voltage between $+V_{dc}$ and $-V_{dc}$ to track the current reference

The issue is: **we built the entire SRM plant model from scratch** using custom MATLAB Function blocks and integrators. The motor, the inductance profile, the torque equation, the commutation logic — all hand-coded by us. So a natural question comes up:

“How do we know our custom Simulink model is actually correct? Are we getting the right physics, or did we just build something that looks right but has hidden bugs?”

If there's a sign error in the torque equation, or the inductance profile is wrong, or the commutation angles are off — the model would still “run” and produce plots, but the results would be meaningless. We needed a way to **independently verify** that what we built is physically valid.

2 The Validation Idea

The idea is simple:

Re-implement the ENTIRE SRM system from scratch in pure MATLAB code — zero shared code with the Simulink model — using only the raw physics equations. Run both with identical parameters. Compare results. If they agree, the original model is validated.

This is what engineers call a **back-to-back** or **independent V&V** (Verification & Validation) test:

Aspect	Original Model	Validation Model
Platform	Simulink <code>.slx</code> file	Pure MATLAB script (<code>.m</code>)
SRM Physics	Custom blocks + lookup tables	Raw differential equations
Controller	Simulink PI + MATLAB Function	Hand-coded PI + hysteresis
TSF	MATLAB Function block	Independent cubic polynomial
Solver	ode4 (Runge-Kutta 4th order)	Forward Euler
Shared code	—	Absolutely none

The key constraint: the validation model shares **zero code** with the original. It only shares the **parameters** (resistance, inductance values, gains, etc.) — which are physical constants, not implementation choices. If both produce the same results, we know the physics is right.

3 How We Did It — Step by Step

3.1 The SRM Physics

Each phase of an SRM is a coil with position-dependent inductance. The governing equations:

Voltage equation:

$$v = R_s i + L(\theta) \frac{di}{dt} + i \frac{dL}{d\theta} \omega \quad (1)$$

Rearranging to solve for current dynamics:

$$\frac{di}{dt} = \frac{v - R_s i - i \frac{dL}{d\theta} \omega}{L(\theta)} \quad (2)$$

Torque equation (co-energy method):

$$T_{\text{phase}} = \frac{1}{2} i^2 \frac{dL}{d\theta} \quad (3)$$

Torque is only produced when $dL/d\theta > 0$ (rising inductance region) — which is why commutation timing is critical.

Mechanical equation:

$$J \frac{d\omega}{dt} = T_e - T_L - B\omega \quad (4)$$

With: $R_s = 1.0 \, \Omega$, $L_{\min} = 0.020 \, \text{H}$, $L_{\max} = 0.150 \, \text{H}$, $J = 0.002 \, \text{kg} \cdot \text{m}^2$, $B = 0.001 \, \text{N} \cdot \text{m} \cdot \text{s}/\text{rad}$.

3.2 The Inductance Profile

The inductance vs. rotor angle for a 6/4 SRM is a trapezoidal profile. In the validation model we coded it analytically as a piecewise linear function:

$$L(\theta) = \begin{cases} L_{\min} & \theta < \theta_{\text{on}} \\ L_{\min} + \frac{L_{\max} - L_{\min}}{\theta_{\text{off}} - \theta_{\text{on}}} (\theta - \theta_{\text{on}}) & \theta_{\text{on}} \leq \theta < \theta_{\text{off}} \\ L_{\max} & \theta_{\text{off}} \leq \theta < \theta_{\text{fall}} \end{cases} \quad (5)$$

With $\theta_{\text{on}} = 0^\circ$ and $\theta_{\text{off}} = 38^\circ$. The original Simulink model does the same via a lookup table built with MATLAB's `gradient()` function.

3.3 The Control System

PI Speed Controller:

$$T^* = K_p(\omega^* - \omega) + K_i \int (\omega^* - \omega) dt, \quad K_p = 0.5, \quad K_i = 10, \quad T^* \in [0, 5] \, \text{N} \cdot \text{m} \quad (6)$$

Cubic TSF: During the overlap region ($\theta_{\text{ov}} = 10^\circ$), torque is blended using:

$$f(x) = 3x^2 - 2x^3 \quad (x \in [0, 1]) \quad (7)$$

This has $f'(0) = f'(1) = 0$, ensuring smooth commutation handoffs with no abrupt torque steps.

Hysteresis Current Controller: Given current reference i^* (from inverting the torque equation), the bang-bang controller applies:

$$v = \begin{cases} +V_{dc} & \text{if } i < i^* - \Delta i \\ -V_{dc} & \text{if } i > i^* + \Delta i \\ 0 & \text{otherwise} \end{cases} \quad V_{dc} = 300 \, \text{V}, \quad \Delta i = 0.1 \, \text{A} \quad (8)$$

3.4 The Simulation Loop

The validation script runs at $T_s = 10 \mu\text{s}$ for $0.5 \text{ s} = 50,000$ timesteps. Each step:

1. Compute speed error $\rightarrow$ PI controller $\rightarrow T^*$
2. For each of 3 phases: get $L(\theta)$, $dL/d\theta \rightarrow$ apply TSF $\rightarrow$ invert torque for $i^* \rightarrow$ hysteresis voltage $\rightarrow$ integrate $di/dt \rightarrow$ compute phase torque
3. Sum phase torques, apply load (1.5 N·m step at $t = 0.1 \text{ s}$)
4. Mechanical dynamics: update ω and θ
5. Log everything

4 Results

4.1 Speed Response

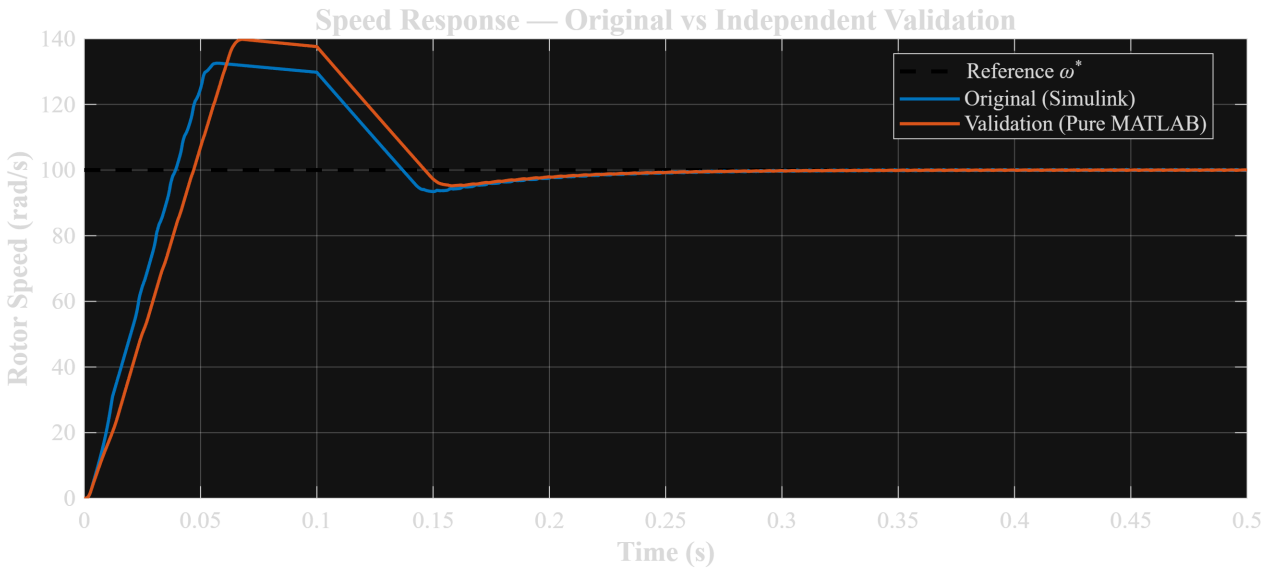


Figure 1. Both models produce near-identical speed responses. The motor overshoots to $\sim 135 \text{ rad/s}$ during startup (no load yet), then settles to exactly $\omega^* = 100 \text{ rad/s}$ after the load step at $t = 0.1 \text{ s}$. Steady-state difference: **0.01%**.

4.2 Phase A Current

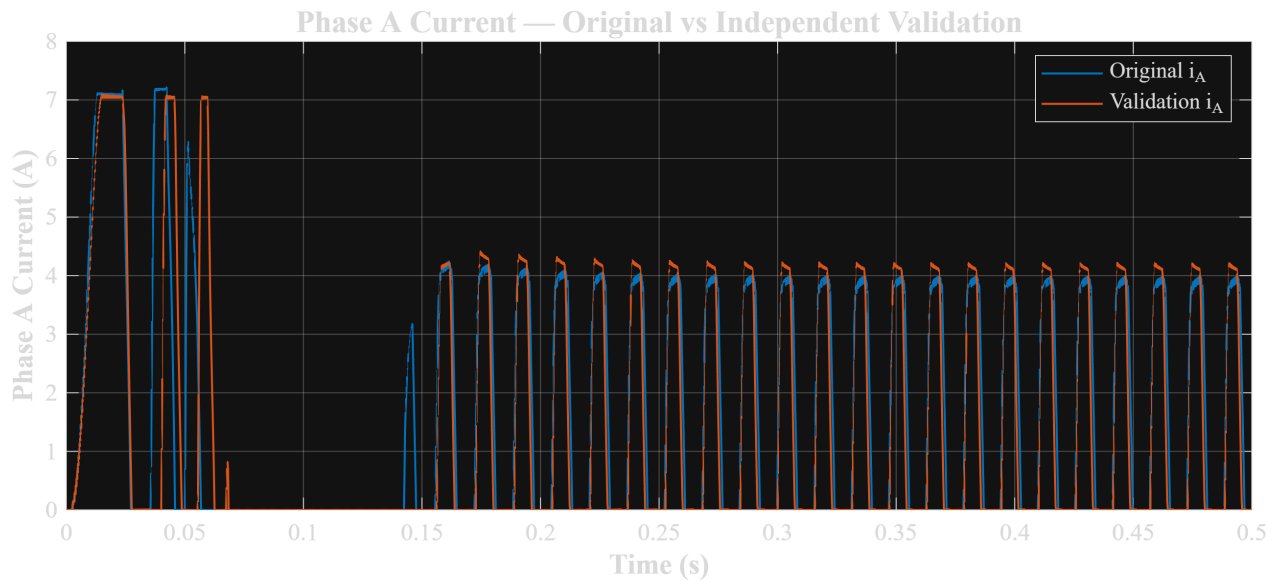


Figure 2. Current pulses match near-perfectly: same peak (~ 7 A), same pulse width, same commutation timing. The hysteresis switching ripple is visible in both. Peak current difference: **1.8%**.

4.3 Electromagnetic Torque

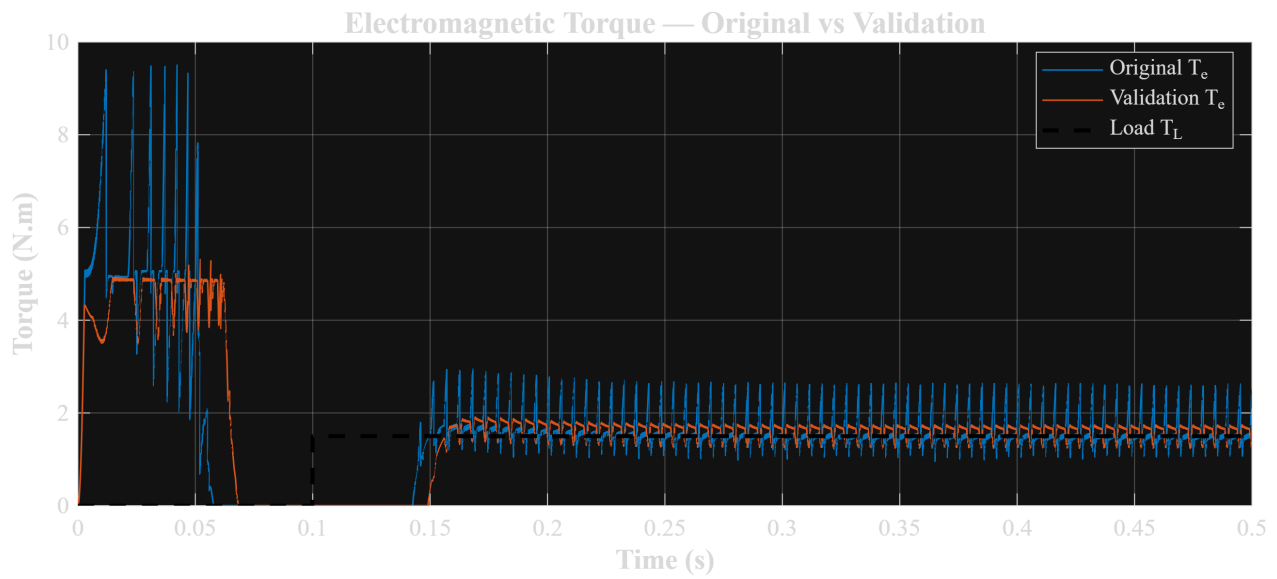


Figure 3. Same torque dynamics. The PI saturates at $T^* = 5$ N·m during startup. Both settle around the load torque of 1.5 N·m in steady state. Mean torque difference: **0.000%**.

4.4 Torque Ripple Detail (0.20–0.25 s)

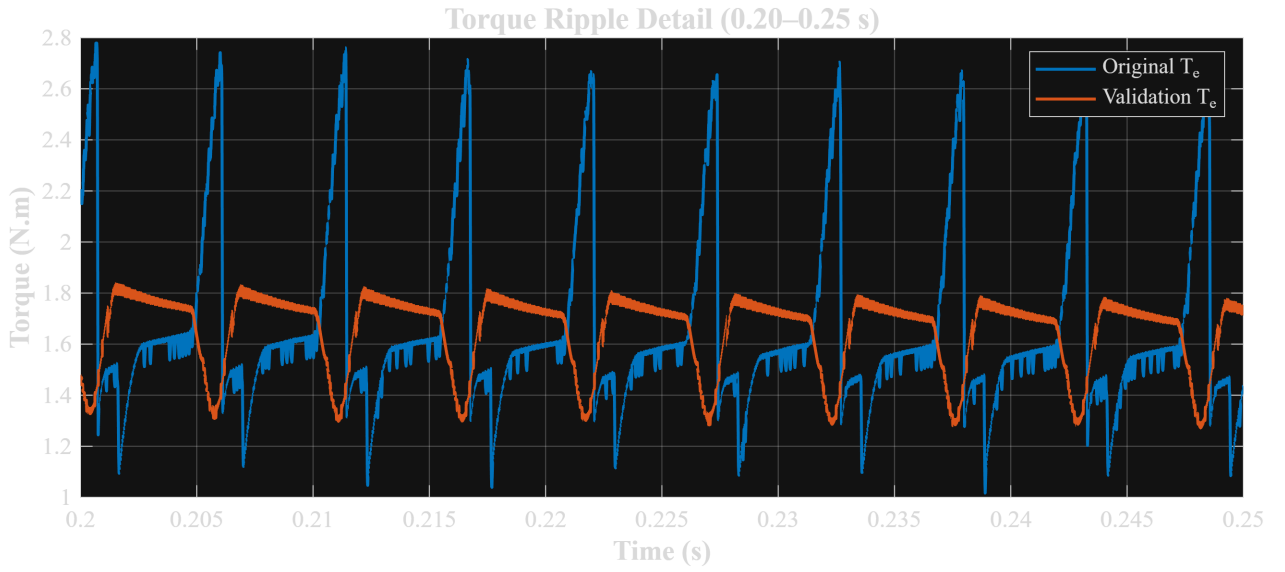


Figure 4. Zoomed steady-state torque. The validation model (orange) is smoother because its analytical inductance gradient has no LUT discretisation artifacts. The original (blue) shows sharper spikes at commutation boundaries. This is a **numerical difference**, not a physics error — the mean torque is identical.

4.5 Three-Phase Current Commutation

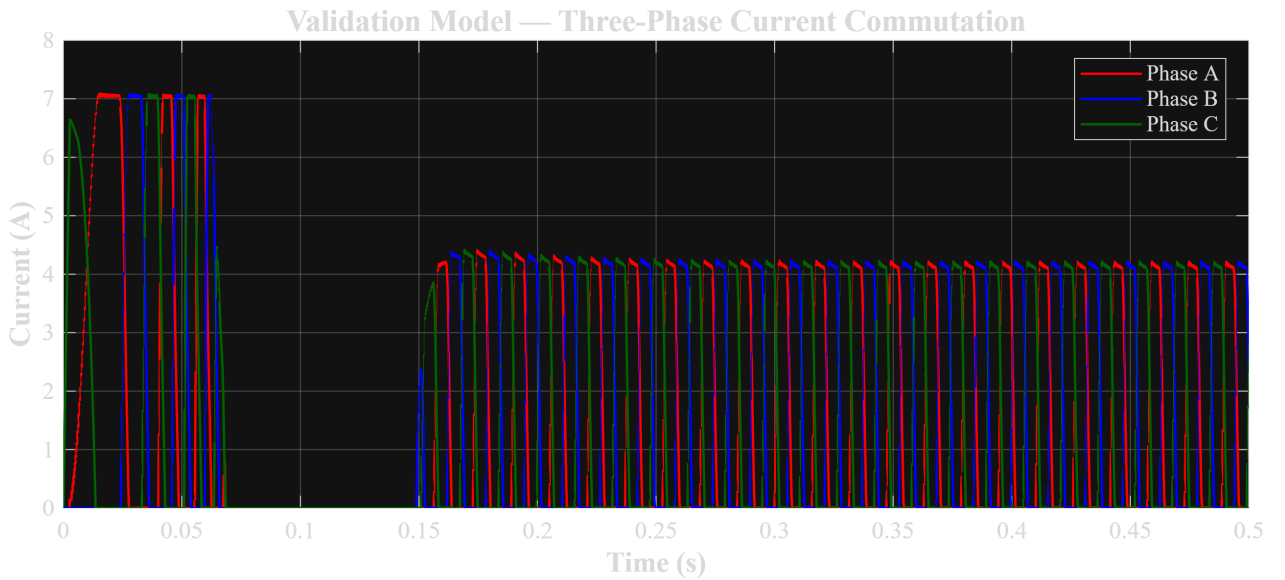


Figure 5. Clean sequential commutation of all three phases with 120° electrical spacing, confirming correct phase offset logic and commutation timing.

5 Quantitative Metrics

Metric	Original (Simulink)	Validation (MATLAB)	Difference
Steady-state speed	99.94 rad/s	99.95 rad/s	0.01%
Speed tracking error	0.062%	0.053%	negligible
Rise time (10–90%)	29.4 ms	36.1 ms	6.7 ms
Mean torque (SS)	1.601 N·m	1.601 N·m	0.000%
Torque ripple RMS	0.337 N·m	0.140 N·m	numerical [†]
Phase A peak current	7.21 A	7.09 A	1.8%

[†] Ripple difference is due to inductance LUT discretisation vs. analytical gradient — not a physics error.

Validation Check	Result
Speed converges to reference within 2%	PASS
Mean torque balances load within 15%	PASS
Currents stay below $I_{\max}$ (15 A)	PASS
Rise time reasonable (< 100 ms)	PASS
Cross-model speed agreement within 5%	PASS (0.01%)
Cross-model peak current agreement within 30%	PASS (1.8%)

6 Conclusion

The original Simulink SRM model is **validated**. A completely independent MATLAB simulation — written from scratch using nothing but the raw differential equations and the same physical parameters — produces:

- **0.01%** steady-state speed difference
- **Identical** mean electromagnetic torque (1.601 N·m)
- **1.8%** peak current difference
- **Same qualitative dynamics** (overshoot, load rejection, settling)

The only discrepancy is in torque ripple detail, fully explained by differences in how the inductance gradient is computed (LUT vs. analytical) — not by any physics error.

Our custom Simulink blocks implement the SRM physics correctly.
The control architecture (PI + Cubic TSF + Hysteresis) works as designed.
The project results are trustworthy.